Document number: P3307R0.
Date: 2024-5-22.
Authors: Gonzalo Brito Gadeschi.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Audience: SG6.

Table of Contents

Floating-Point Maximum/Minimum Function Objects

Adds floating-point aware minimum/maximum function objects following C23 APIs to the `<functional>` header.

Motivation

Maximum/minimum parallel reductions - e.g. via `std::reduce` or `std::transform_reduce` - are very common in scientific applications, e.g., to determine the admisible time-step during explicit time integration of partial-differential equations. Modern data center hardware provides hardware acceleration for these sort of reductions.

While C++ provides such function objects via `std::ranges::min` / `::max`, these function objects exhibit undefined behavior in the presence of NaNs, and *require* `-0 == +0` (i.e. `-0 < +0` to be false), which goes against IEEE 754 standard recommendations, and prevents hardware acceleration that follows these recommendations from being used.

Before

```
#include <cassert>
#include <execution>
#include <numeric>
#include <ranges>

int main() {
  double data[] = {2, 1, 4, 3};
  double max = std::reduce(
    std::execution::par,
    data, data + 5,
    0,
    // Does not use HW accel
    std::ranges::max
  );
  assert(max == 4);
  return 0;
}
```

After

```
#include <cassert>
#include <execution>
#include <functional>
#include <numeric>

int main() {
  double data[] = {2., 1, 4, 3}
  double max = std::reduce(
    std::execution::par,
    data, data + 5,
    0,
    // Uses HW accel
    std::fmaximum_num_t{}
  );
  assert(max == 4);
  return 0;
}
```

Why can't users work-around this? Because floating-points are not user-defined types, so users can't fix their `operator<`.

Design Alternatives

We add 6 function objects to `<functional>`, 4 of which correspond to the 4 new C23 floating-point minimum and maximum APIs: `std::fminimum`, `std::fmaximum`, `std::fminimum_num`, `std::fmaximum_num`, and 2 of which correspond to `std::fmin` and `std::fmax`. The semantics of these function objects for floating-point types are "calls the corresponding C API".

Design constraints:

- names can't collide with the C23 API names.

Design alternatives

- Which types `T` to support?
  1. Only `is_floating_point_v<T>` is true.
  - PRO: simple constraint
  - CON: less convenient for generic code that needs to handle non-floating-point types

2. `is_floating_point_v<T>` is true or meets `Cpp17LessThanComparable` requirements.
   - PRO: more convenient for generic code that needs to handle non-floating-point types
   - CON: more complex constraint, does not map to any pre-existing concepts.

- Naming scheme to avoid collisions
  1. Use a `_t` underscore
  - CONs: not a type alias, extra typing, etc.
  2. Drop the `f`
  - CONs: does not work for `fmin` / `fmax` which would collide with `std::min` / `std::max`

Proposed design: use `_t` to disambiguate and do not support non floating-point types.

```
namespace std {
   struct fmin_t;
   struct fmax_t;
   struct fminimum_t;
   struct fmaximum_t;
   struct fminimum_num_t;
   struct fmaximum_num_t;
}
```

Wording

TBD.